Get unlimited access to the best of Medium for less than $1/week. Become a member

Creating and Deploying Packages on vSphere Kubernetes Service: An Addon Management Guide

Navneet Verma Follow 13 min read · Nov 17, 2025

2 1

Introduction

vSphere Kubernetes Service (VKS) leverages Carvel, a robust set of Kubernetes tools, to provide a declarative approach for packaging, distributing, and installing applications. VKS 3.5 provides a new mechanism for managing add-ons in VKS Clusters.

The new Add-on Management feature allows you to install, upgrade, and configure standard packages. It offers several advantages over previously available methods for installing add-ons, including:

- The package repository is automatically installed and upgraded by VKS
- Provides a single method for installing and upgrading all **standard packages.**

- Provides a **declarative** API for add-on installation and configuration, enabling add-on management across the fleet of workload clusters.
- Allows pre-checks to be run before add-on upgrades and cluster upgrades, ensuring that these upgrades do not break any add-on dependencies or compatibility constraints.

This unique declarative approach to add-on deployment across multiple clusters in a cloud-native manner, the author explored whether this process could also be leveraged to deploy and manage non-VMware-provided add-ons on VKS clusters. It led to this write-up.

In this comprehensive guide, we'll walk through the entire process of:

1. Creating a Carvel package from a sample Helm chart

2. Building a Carvel package repository that will include the Carvel package and can be made available for general consumption.

3. (Optional) Since we built a package repository, why not validate its basic workflow on a VKS cluster

4. Configuring Add-on resources on the Supervisor cluster to leverage the Carvel package repository built in Step 2.

5. Installing the package created in Step 1 on a VKS workload cluster using Add-on Management

This tutorial uses Trivy Operator as our example application, but the process applies to any Kubernetes application you want to package and distribute.

Prerequisites

Before we begin, ensure you have the following tools installed:

`kctrl` — Carvel package controller CLI

`ytt` — YAML templating tool

`kbld` — Image building tool

`imgpkg` — Image packaging tool

`kapp` — Application deployment tool

`helm` — Helm CLI (for working with Helm charts)

`kubectl` — Kubernetes CLI

`vcf` — VMware Cloud Foundation CLI (for VKS operations)

Access to:

- A VKS Supervisor cluster with VKS Service 3.5.0+
- A VKS workload cluster (for validation).
- An OCI registry (Docker registry, GCR, etc.) with push permissions
- Access to the following GitHub repo that contains the sample scripts and manifests that were used to write this article. https://github.com/papivot/vks-package-scaffolding.git. Clone the repo in

your local environment. The scripts can be modified as per users' environments.

- Sample Trivy-operator helm chart values.yaml file.

Part 1: Creating a Carvel Package from a Helm Chart

Step 1: Scaffold the Package Structure

We'll use a scaffolding script to create the initial package structure. The script makes all necessary directories and sample files for a VKS package.

```
$ ./vks-package-scaffold.sh trivy-operator 0.0.6 my-awesome-registry.com/vks-pac
```

This command creates:

- Package directory structure
- Configuration files for ytt templating
- Sample overlay files
- Package build and resource definitions

What the script does:

- Creates directories: `distribution/`, `config/_ytt_lib/bundle/config/upstream/`, and `config/_ytt_lib/bundle/config/overlays/`
- Generates `package-build.yml` for building the package

- Generates `package-resources.yml` for package metadata
- Creates a sample namespace overlay with Pod Security Admission (PSA) labels

This is how the directory structure looks after scaffolding —

```
├── trivy-operator
│   ├── config
│   │   ├── _ytt_lib
│   │   │   └── bundle
│   │   │       └── config
│   │   │           ├── overlays
│   │   │           └── upstream
│   │   ├── config.yaml
│   │   └── values.yaml
│   ├── distribution
│   │   ├── sample-overlay-namespace.yaml
│   │   └── sample-namespace.yaml
│   ├── package-build.yml
│   └── package-resources.yml
├── vks-package-repo.sh
└── vks-package-scaffold.sh
```

Step 2: Extract Helm Chart Manifests

Navigate to the distribution directory and extract the Helm chart:

```
$ cd trivy-operator/distribution

# Add the Helm repository
$ helm repo add aqua https://aquasecurity.github.io/helm-charts/
$ helm repo update

# Download the default values
$ helm show values aqua/trivy-operator > values.yaml

# Customize values as needed (optional)
```

```
# Edit values.yaml or create trivy-operator-values.yaml
# Generate Kubernetes manifests from Helm chart
$ helm template trivy-operator aqua/trivy-operator -n trivy-operator -f trivy-op
```

Open in app ↗

Medium Search Write S

- `--include-crds` ensures that CustomResourceDefinitions (if any) are included in the downloaded manifest.
- The `-f` flag allows you to override default values with the customizations that we would like to implement on the application deployment.
- The output is a templated YAML file **ready** for Kubernetes deployment.

Step 3: Add a Namespace with Pod Security Admission

VKS clusters require Pod Security Admission (PSA) labels on namespaces. The scaffolding script creates a sample namespace file. Append it to your generated manifest:

```yaml
# The sample-namespace.yaml contains:
apiVersion: v1
kind: Namespace
metadata:
  name: trivy-operator
  labels:
    pod-security.kubernetes.io/enforce: privileged
    pod-security.kubernetes.io/enforce-version: latest
    pod-security.kubernetes.io/audit: privileged
    pod-security.kubernetes.io/audit-version: latest
    pod-security.kubernetes.io/warn: privileged
    pod-security.kubernetes.io/warn-version: latest
---
```

```
cat sample-namespace.yaml >> trivy-operator.yaml
```

Step 4: Move Manifests to Upstream Directory

Perform any additional modifications to the generated manifests and copy them to the upstream directory where `ytt` will process them:

```
$ cp trivy-operator.yaml ../config/_ytt_lib/bundle/config/upstream/
$ cd ..
```

During this stage, we may need to create some YTT overlays in the config/overlays folder. These overlays are meant to customize the manifest during the deployment stage, based on the configuration provided to the Carvel package. This topic is currently out of scope for this article, as we purposely selected/built a manifest file that is static and does not require any further customization during the installation process.

Step 5: Initialize and Release the Package

Now we'll use `kctrl` to initialize and release the package:

```
# Initialize the package (creates/updates package-build.yml).
# Select the defaults, as most of the parameters where generated at the scaffold
$ kctrl package init

Welcome! Before we start, do install the latest Carvel suite of tools,
specifically ytt, imgpkg, vendir and kbld.

Basic Information
A package reference name must be at least three '.' separated segments,e.g.
samplepackage.corp.com
> Enter the package reference name (trivy-operator.fling.vsphere.vmware.com):

Content
Please provide the location from where your Kubernetes manifests or Helm chart
can be fetched. This will be bundled as a part of the package.
1: Local Directory
```

```
2: Github Release
3: Helm Chart from Helm Repository
4: Git Repository
5: Helm Chart from Git Repository
> Enter source (1):

We need to include files/ directories which contain Kubernetes manifests.
Multiple values can be included using a comma separator.
> Enter the paths which contain Kubernetes manifests (./config):

Output
Successfully updated package-build.yml
Successfully updated package-resources.yml

Next steps
Created files can be consumed in following ways:
1. `package release` command to release the package.
2. `package release --repo-output repo` to release the package and add it to the
package repository directory.

Succeeded
```

What happens during the initialization process:

1. The files `package-build.yml` and `package-resource.yml` get updated with the values provided during the init command.

```
# Release the package with OpenAPI schema and validations
$ kctrl package release --openapi-schema --version 0.0.6 --build-ytt-validations

Prerequisites
1. Host is authorized to push images to a registry (can be set up by running
`docker login`)
2. `package init` ran successfully.

The bundle created needs to be pushed to an OCI registry. (format:
<REGISTRY_URL/REPOSITORY_NAME>) e.g. index.docker.io/k8slt/sample-bundle
> Enter the registry URL (my-awesome-registry.com/vks-package/trivy-operator):

kbld builds images when necessary and ensures that all image references are
resolved to an immutable reference
```

```
Building images and resolving references
     | $ ytt -f /var/folders/4_/x04767093bq3r5lkn0tnl1pw0000gq/T/kapp-controller
     | $ kbld -f - --imgpkg-lock-output=.imgpkg/images.yml

An imgpkg bundle consists of all required manifests bundled into an OCI image.
This image is pushed to a registry and consumed by the package.
Pushing imgpkg bundle
     | $ imgpkg push -b my-awesome-registry.com/vks-package/trivy-operator:build
     | dir: .
     | dir: .imgpkg
     | file: .imgpkg/images.yml
     | dir: config
     | dir: config/_ytt_lib
     | dir: config/_ytt_lib/bundle
     | dir: config/_ytt_lib/bundle/config
     | dir: config/_ytt_lib/bundle/config/overlays
     | dir: config/_ytt_lib/bundle/config/upstream
     | file: config/_ytt_lib/bundle/config/upstream/trivy-operator.yaml
     | file: config/config.yaml
     | file: config/values.yaml
     | Pushed 'my-awesome-registry.com/vks-package/trivy-operator@sha256:cf5d24e
     | Succeeded
Artifact created: carvel-artifacts/packages/trivy-operator.fling.vsphere.vmware.
Artifact created: carvel-artifacts/packages/trivy-operator.fling.vsphere.vmware.

Next steps
1. The artifacts generated by the `--repo-output` flag can be bundled into a
PackageRepository by using the `package repository release` command.
2. Generated Package and PackageMetadata manifests can be applied to the cluster
directly.

Succeeded
```

What happens during release:

1. `kctrl` processes the YAML files through ytt templating
2. Builds any container images (if needed)
3. Creates an imgpkg bundle with all package contents
4. Upload the bundle to a container registry
5. Generates `Package` and `PackageMetadata` resources

6. Outputs artifacts to `carvel-artifacts/packages/`

Verify the release:

```
$ ls -1 carvel-artifacts/packages/trivy-operator.fling.vsphere.vmware.com/

# Should show:
# metadata.yml
# package.yml
```

This is the current directory structure after this step is completed.

```
├── trivy-operator
│   ├── carvel-artifacts
│   │   └── packages
│   │       └── trivy-operator.fling.vsphere.vmware.com
│   │           ├── metadata.yml
│   │           └── package.yml
│   ├── config
│   │   ├── _ytt_lib
│   │   │   └── bundle
│   │   │       └── config
│   │   │           ├── overlays
│   │   │           │   └── # [[some ytt overlays as needed. none in this example
│   │   │           └── upstream
│   │   │               └── trivy-operator.yaml
│   │   ├── config.yaml
│   │   └── values.yaml
│   ├── distribution
│   │   ├── sample-overlay-namespace.yaml
│   │   ├── sample-namespace.yaml
│   │   └── trivy-operator.yaml
│   ├── package-build.yml
│   └── package-resources.yml
├── vks-package-repo.sh
└── vks-package-scaffold.sh
```

Step 6: Modify the Package YAML for Addon Management

Since the goal of this exercise is for the newly created package to be deployed by the Addon management via the Supervisor, we need to annotate the newly created `package.yml` with a valid annotation such that Addon Controller can reconcile the package from the package repository and generate the corresponding Addon and AddonRelease objects (see Part 4 below).

To do, edit the file `trivy-operator/carvel-artifacts/packages/trivy-operator.fling.vsphere.vmware.com/pakage.yml` and add the following annotation —

```
addons.kubernetes.vmware.com/addon-config-definition: “”
```

The file should look something like this —

```yaml
apiVersion: data.packaging.carvel.dev/v1alpha1
kind: Package
metadata:
  creationTimestamp: null
  name: trivy-operator.fling.vsphere.vmware.com.0.0.6
  annotations:
    addons.kubernetes.vmware.com/addon-config-definition: ""
spec:
  refName: trivy-operator.fling.vsphere.vmware.com
...
...
```

Note:

Since we are not adding any specific configuration or customization to this package, an empty string is sufficient. We will discuss the annotation and its use in more detail in a later blog.

Part 2: Creating a Carvel Package Repository

A package repository is a collection of packages that can be distributed and installed on clusters. Let's create one such repository that includes our Trivy-operator package.

Step 1: Build the Repository Structure

Use the repository script to create the repository structure:

```
$ ./vks-package-repo.sh vks-custom-packages trivy-operator
```

What the script does:

1. Creates repository directory structure
2. Copies the Trivy-operator `metadata.yml` to the repository's packages folder (resulting output of Step 1)
3. Extracts version from `package.yml` and copies it as `{version}.yml` to the repository's packages folder (resulting output of Step 1)
4. Uses `kbld` to lock image references
5. Generates `.imgpkg/images.yml` for image locking

See the directory structure below:

```
.
├── trivy-operator
│   ├── carvel-artifacts
│   │   └── packages
│   │       └── trivy-operator.fling.vsphere.vmware.com
│   │           ├── metadata.yml
│   │           └── package.yml
...
...
├── vks-custom-packages
│   └── .imgpkg
│       └── images.yml
│   └── packages
│       └── trivy-operator.fling.vsphere.vmware.com
│           ├── 0.0.6.yml
│           └── metadata.yml
├── vks-package-repo.sh
└── vks-package-scaffold.sh
```

Step 2: Push Repository to Registry

Push the repository bundle to your OCI registry:

```
$ imgpkg push -b my-awesome-registry.com/vks-package/vks-custom-packages:0.0.1 -
```

Part 3: Validating the Package on VKS Cluster

Before configuring the Supervisor, let’s validate that our package works correctly on a VKS cluster.

Step 1: Add Package Repository to Cluster

Log in to a VKS cluster where you would like to test the package deployment and run the following commands —

```
$ vcf package repository add vks-custom-packages --url my-awesome-registry.com/w
```

Step 2: Verify Packages Are Available

List available packages:

```
$ vcf package available list -n tkg-system -o wide
```

You should see `trivy-operator.fling.vsphere.vmware.com` in the list.

Step 3: Install the Package

Install the package on the cluster:

```
$ vcf package install trivy-pkg -n tkg-system -package trivy-operator.fling.vsph
```

Step 4: Verify Installation

Check the installation status:

```
$ vcf package installed list -n tkg-system
# Expected output:
# NAME PACKAGE-NAME PACKAGE-VERSION STATUS
# trivy-pkg trivy-operator.fling.vsphere.vmware.com 0.0.6 Reconcile succeeded
```

Verify pods are running:

```
$ kubectl get pods -n trivy-operator

# Expected output:
# NAME                            READY      STATUS   RESTARTS   AGE
# trivy-operator-6475b58bc4-vqzhz 1/1        Running  0          6m6s
# trivy-server-0                  1/1        Running  0          6m6s
```

Step 5: Cleanup (Optional)

If you want to clean up the test installation:

```
# Delete the package installation
$ vcf package installed delete -n tkg-system trivy-pkg

# Delete the package repository
$ vcf package repository delete vks-custom-packages -n tkg-system
```

Part 4: Configuring Addon Resources on Supervisor

This is the section where all the magic happens. Note that this feature is currently experimental. As stated earlier, the purpose of this article is to demonstrate the art of the possible. ***While this example leverages a Trivy Operator manifest, this method can be used to deploy and manage any user-specific add-ons (or Kubernetes manifests — e.g., custom webhooks, network policies, etc.) in a GitOps fashion at the Supervisor level. This could help enforce standard software and policy deployments on VKS clusters.***

Step 0: Modify the VKS Service setting

Once validated, we need to configure the Supervisor cluster to make this package available as an Add-on that can be installed on VKS workload clusters through the new Add-on management (introduced in VKS 3.5). The

base VKS service comes with a default repository. To add new Add-on package repositories, we must add the name of our repo to the allow list. This can be done by updating the VKS Service configuration within the vCenter UI.

- Navigate to Supervisor/Workload Management -> Supervisors
- Select the Supervisor with the VKS service 3.5, where the new configuration will be updated, and navigate to Supervisor Services (center pane) -> Overview.
- Select Kubernetes Service -> Manage and navigate to the screen that shows the *YAML Service Config (optional)*
- **Add** the following value in a new line (**Do not modify the existing configuration i.e.** `namespace: svc-tkg-domain-cxxx`)

```
addonRepoAllowlist:
- custom-packages
```

- Note the name of the repo that has been provided — `custom-packages` In the above example. Click Finish to complete the configuration.
- This updates the `addon-repo-allow-list-configmap` ConfigMap within the Supervisor.

Step 1: Create AddonRepository

Create a new `AddonRepository`. Notice that the `targetRepositoryName` should match the value in the addonRepoAllowlist that was created in Step 0.

The `AddonRepository` resource tells the Supervisor where to find your package repository and which packages/versions to expose.

```yaml
apiVersion: addons.kubernetes.vmware.com/v1alpha1
kind: AddonRepository
metadata:
  name: custom-addonrepository-0.0.1
  namespace: vmware-system-vks-public
  annotations:
    addons.kubernetes.vmware.com/package-offerings: |
      {
        "repositoryVersion": "0.0.1",
        "packages": {
          "trivy-operator.fling.vsphere.vmware.com": {
            "versions": ["0.0.6"]
          }
        }
      }
spec:
  fetch:
    imgpkgBundle:
      imageURL: my-awesome-registry.com/vks-package/vks-custom-packages:0.0.1
  targetRepositoryName: custom-packages
  version: 0.0.1
---
apiVersion: addons.kubernetes.vmware.com/v1alpha1
kind: AddonRepositoryInstall
metadata:
  name: custom-addon-repo-install
  namespace: vmware-system-vks-public
  annotations:
    addons.kubernetes.vmware.com/target-repository-name: custom-addonrepository-
spec:
  addonRepositoryRef:
    name: custom-addonrepository-0.0.1
    namespace: vmware-system-vks-public
```

Key components:

- **`package-offerings` annotation:** Specifies which packages and versions to expose
- **`imageURL`** : Points to your repository bundle in the registry
- **`targetRepositoryName`** : Name for the PackageRepository created on the Supervisor. Should match the value created in Step 0 above.
- **`AddonRepositoryInstall`** : Triggers the installation of the repository

Apply the configuration:

```
$ kubectl apply -f supervisor-addonrepository.yaml
```

Step 2: Verify Packages Are Available

After the AddonRepository is installed, verify that the Trivy-operator package is available in the Supervisor:

```
$ kubectl get pkg -n vmware-system-vks-public
# You should see:
NAME                                                PACKAGEMETADATA NAME
trivy-operator.fling.vsphere.vmware.com.0.0.6     trivy-operator.fling.vsphere.v
```

Step 3: Validate Addon and AddonRelease

If the annotations were correctly applied in Part 1/Step 6, the objects listed below should have been created by the Addon controller.

```
$ kubectl  get addon -n vmware-system-vks-public
# You should see:
NAME                          AGE
...
trivy-operator                21m
...

$ kubectl get AddonRelease -n vmware-system-vks-public
# You should see:
NAME                                                                  ADDON
...
trivy-operator.fling.vsphere.vmware.com.0.0.7                         trivy-
...
```

We are now ready to deploy the Trivy Operator to our VKS clusters!!!

Part 5: Installing Addon on VKS Workload Clusters

Now that the Add-on is configured on the Supervisor, you can install it on VKS workload clusters declaratively.

Step 1: Create AddonInstall

The `AddonInstall` resource specifies which clusters should have the Add-on installed:

```
apiVersion: addons.kubernetes.vmware.com/v1alpha1
kind: AddonInstall
metadata:
  name: cluster-trivy
  namespace: demo1 # Your vSphere namespace
spec:
  addonRef:
    name: trivy-operator
  clusters:
  - selector:
```

```
        matchLabels:
          cluster.x-k8s.io/cluster-name: workload-vsphere-vks2
```

Key points:

- `metadata.namespace` : The vSphere namespace where your VKS clusters are located
- `spec.addonRef.name` : Must match the Addon name from Part 4
- `spec.clusters.selector` : Uses cluster labels to select target clusters

Apply the configuration:

```
$ kubectl apply -f addon-install.yaml
```

Step 2: Verify Installation

Once you deploy a VKS cluster named `workload-vsphere-vks2` in the `demo1` namespace or the VKS cluster with that name already exists in the vSphere Namespace, the Trivy Operator will be automatically installed in the VKS using the Add-on Management and the Kapp Controller.

Verify on the workload cluster:

```
# Connect to the workload cluster
$ kubectl get pods -n trivy-operator
```

```
# Check PackageInstall status
$ kubectl get packageinstall -n trivy-operator
```

Best Practices and Tips

1. Version Management

- Use semantic versioning (e.g., 0.0.6, 1.0.0)
- Update the `package-offerings` annotation when adding new versions
- Keep older versions available for rollback scenarios

2. Namespace Configuration

- Always include Pod Security Admission labels
- Use consistent namespace naming across packages
- Consider namespace lifecycle management

3. Image Locking

- Always use `kbld` to lock image references
- This ensures reproducible deployments
- The `.imgpkg/images.yml` file tracks all images

4. Testing Workflow

- Test locally with `kctrl dev` (if using local development)
- Validate on a test VKS cluster

- Configure on Supervisor
- Deploy to production workload clusters

5. Registry Organization

- Use a consistent naming convention: `{registry}/{project}/{repo-name}:{version}`
- Tag repositories with semantic versions
- Keep repository bundles immutable

Troubleshooting

Package Installation Fails

- Check Pod Security Admission labels on the namespace
- Verify image pull secrets are configured
- Review PackageInstall status: `kubectl describe packageinstall -n <namespace>`

Addon Not Appearing

- Verify AddonRepository is installed: `kubectl get addonrepository -n vmware-system-vks-public`
- Check the package offerings annotation matches available packages
- Ensure `PackageMetadata` and `Package` resources exist

Images Not Pulling

- Verify registry authentication
- Check `.imgpkg/images.yml` for correct image references

- Ensure workload clusters have access to the registry

Conclusion

This guide has walked you through the complete lifecycle of creating, packaging, and deploying applications on VKS using Carvel:

1. **Package Creation:** Convert Helm charts to Carvel packages
2. **Repository Building:** Create distributable package repositories
3. **Validation:** Test packages on VKS clusters before Supervisor configuration
4. **Supervisor Configuration:** Expose packages as Add-ons
5. **Workload Deployment:** Automatically install Add-ons on VKS clusters

For more information:

- Carvel Documentation
- vSphere Kubernetes Service Documentation
- Carvel GitHub Repository

Vks Kubernetes Vmware Vsphere Vcf Vsphere With Tanzu

Written by Navneet Verma

106 followers · 5 following

Follow

Responses (1)

S Sridhar Kumardevo

What are your thoughts?

N Rajeshnadar
Jan 8

Hi Navneet,

This document is very useful—thank you for sharing it.

I have a question regarding converting Helm packages to Carvel packages. Are we gaining any specific advantages by doing this? Also, are Carvel packages compatible with platforms other... more

Reply

More from Navneet Verma

Navneet Verma

Headlamp on VKS: OIDC, Security Plugins, and Lessons Learned

Introduction

Mar 14 1

Navneet Verma

No-Nonsense Guide: Deploying Headlamp on VKS (Helm + Plugins)

Introduction

Sep 14, 2025 5 1

Navneet Verma

Installing and Initializing Ubuntu VMs using Cloud-init on vSphere 7.

This article demonstrates how to deploy Ubuntu VMs within a vSphere 7 environment...

Nov 22, 2020 8 2

Navneet Verma

Mastering Security: A Guide to Installing and Operating Trivy...

Why You Should Care About Trivy Operator on VKS

Apr 17, 2025 1 1

See all from Navneet Verma

Recommended from Medium

Goran Jovanov

From One Box to Three: Building a Multi-Host HA Kubernetes Cluste...

A guide on evolving from a single-server K8s setup to a 3-host, 6-VM, WireGuard-meshed...

Mar 7

Agent Native

Local LLMs That Can Replace Claude Code

Editor's note: While hardware tiers remain valuable context for this article, we've...

Jan 20 1.8K 52

Kristi Ndoni

PostgreSQL migration with CloudNativePG Logical Replicatio...

This article provides a comprehensive guide to installing CloudNativePG (CNPG) on a...

Dec 16, 2025 3

Ogonna Nnamani

Demystifying The Networking Behind The GitHub Self-Hosted...

I recently deployed GitHub Actions Runner Controller (ARC) on Google Kubernetes...

Jan 31 48

Sujith Samuel

Scaling Machine Learning and Simulation Workloads with Multi-...

In the world of machine learning and large-scale simulations, running everything in a...

Nov 23, 2025 58 1

Salwan Mohamed

Cluster API from Day-0: The Platform Engineering Decision Yo...

Why Your First Cluster Defines Your Last Cluster—And How to Get It Right

Dec 15, 2025 18

See more recommendations